ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Modelli LiteRT-based per Machine Learning Efficiente su Dispositivi Edge

Tesi di Laurea di Andrea Passini
Relatore: Prof. Paolo Bellavista

Anno Accademico 2024-25

Machine Learning distribuito - AI on-device

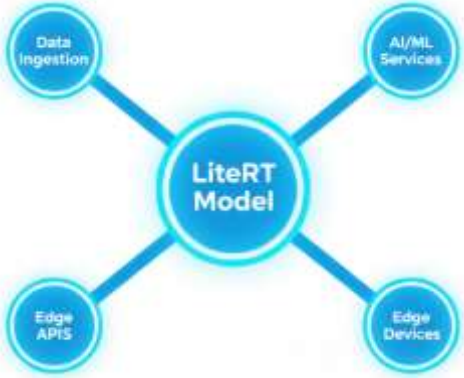
Intelligenza artificiale eseguita sul dispositivo periferico.

Latenza ridotta, Resilienza, Personalizzazione, Scalabilità orizzontale

LiteRT

Runtime modulare, ultraleggero, alte prestazioni, sviluppato da Google.

Nuovo paradigma - Interoperabilità multi-framework



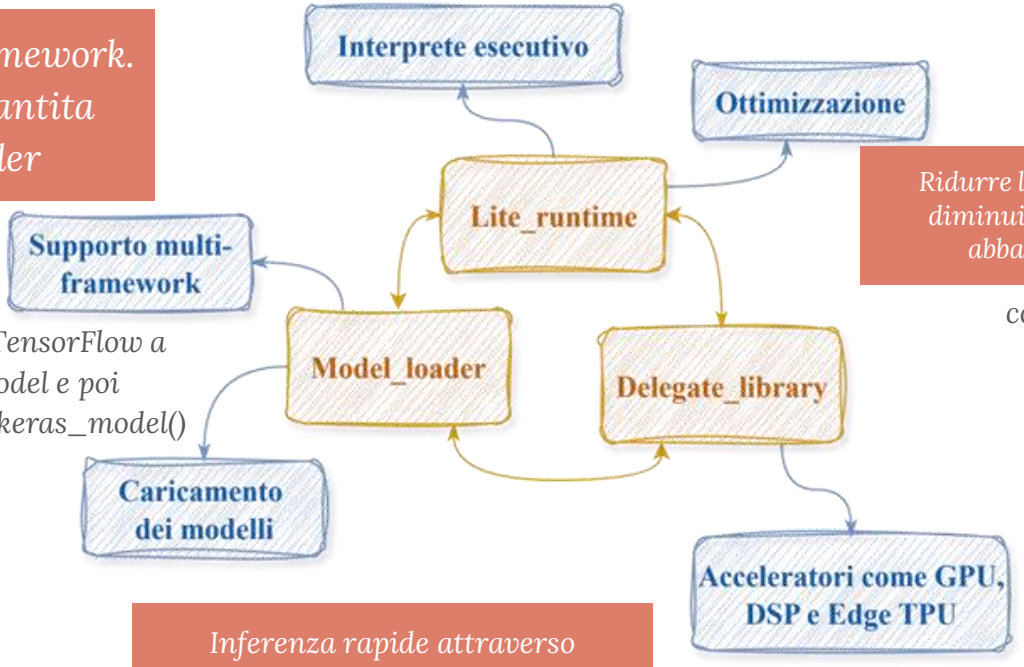
Modello come nodo centrale per logica "hub-and-spoke"

Architettura di LiteRT

Runtime modulare con l'obiettivo di offrire un'elevata flessibilità e adattabilità in ambienti edge

new Interpreter in ambiente di sviluppo

Approccio multi-framework. Compatibilità garantita dal model_loader



Ridurre la complessità computazionale, diminuire la dimensione del modello e abbassare i consumi energetici.

converter.optimizations a default

API classica per passare da TensorFlow a TFLite: Formato SavedModel e poi tf.lite.TFLiteConverter.from_keras_model()

Inferenza rapide attraverso operazioni tensoriali in parallelo

API TFLiteGpuDelegate

Caso di studio - Visione Artificiale

Rilevamento di immagini

Github YOLOv5 - Modello pre-addestrato sul dataset COCO

SavedModel

TFLite

Preprocessing

Inferenza

Postprocessing

Rendering Visivo

Rilevazione di Anomalie

Autoencoder

Preparazione del dataset MVTec AD di Paul Bergmann, Michael Fauser, David Sattlegger, e Carsten Steger

Addestramento

Valutazione delle Prestazioni

TFLite

Inferenza

Rilevamento di Immagini

```java
// Carica il file .tflite dal file system
private MappedByteBuffer loadModelFile(AssetManager assetManager, String modelPath) throws IOException {
    AssetFileDescriptor fileDescriptor = assetManager.openFd(modelPath);
    FileInputStream inputStream = new FileInputStream(fileDescriptor.getFileDescriptor());
    FileChannel fileChannel = inputStream.getChannel();
    long startOffset = fileDescriptor.getStartOffset();
    long declaredLength = fileDescriptor.getDeclaredLength();
    return fileChannel.map(FileChannel.MapMode.READ_ONLY, startOffset, declaredLength);
}
```

```java
fun runInference(onFinished: (Bitmap?) -> Unit) {  1 Usage
    viewModelScope.launch { this: CoroutineScope
        checkInference = false
        val result: Pair<Bitmap, List<Detection>>? = withContext( context = Dispatchers.Default) { this: CoroutineScope
            // Esegui l'inferenza e ottieni il bitmap annotato e la lista di rilevazioni
            inferenceModel.detect(selectedBitmap,  detectionThreshold = threshold,  classifier = classifier!!)
        }
```

```java
// Ridimensiona l'immagine a 640x640
Bitmap resizedBitmap = Bitmap.createScaledBitmap(bitmap, INPUT_SIZE, INPUT_SIZE,  filter: true);

// Converte il bitmap in input ByteBuffer (formato float32 RGB normalizzato [0,1])
ByteBuffer inputBuffer = convertBitmapToByteBuffer(resizedBitmap);

// Alloca array di output della dimensione corretta per YOLOv5
float[][][] output = new float[1][OUTPUT_ROWS][OUTPUT_COLUMNS];

// Esegue inferenza
tflite.run(inputBuffer, output);
```

API: Interpreter TensorFlow Lite

INFERENZA

Il tensore grezzo è rappresentato dall'array float[][][] output, che viene popolato dal metodo tflite.run().

Rilevamento di Immagini - ApplyNMS()

```java
// Funzione per applicare il Non-Maximum Suppression (NMS)
public static List<Detection> applyNMS(List<Detection> detections, float iouThreshold) { 1 usage
    // Ordinare per confidenza
    Collections.sort(detections, ( Detection d1,  Detection d2) -> Float.compare(d2.objectness, d1.objectness));

    List<Detection> filteredDetections = new ArrayList<>();

    while (!detections.isEmpty()) {
        // Prendi la previsione con la confidenza più alta
        Detection bestDetection = detections.remove( index: 0);
        filteredDetections.add(bestDetection);

        // Filtra le detections che si sovrappongono troppo (IoU > soglia)
        detections.removeIf( Detection detection -> computeIoU(bestDetection, detection) > iouThreshold);
    }

    return filteredDetections;
}
```

soglia predefinita del 70%, il riquadro con la confidenza più bassa viene scartato in favore di quello con la confidenza più alta.

In fase di rendering sono validi solamente i rettangoli che hanno superato anche il controllo di Non-Maximum Suppression

Limite: L'algoritmo di Non-Maximum Suppression (NMS) implementato è di tipo single-class e utilizza un valore di intersezione fisso al 70%, deciso in fase di progettazione

Rilevamento di Immagini - Ottimizzazioni

Quantizzazione INT8

Riduce la precisione e aumenta notevolmente la velocità e la stabilità del sistema, convertendo i valori dei numeri per pesi e attivazioni da virgola mobile a 16 bit a interi a 8 bit.

```python
def convert_to_tflite_int8():
    # TensorFlow Lite Converter API
    converter = tf.lite.TFLiteConverter.from_saved_model(saved_dir)
    # Ottimizzazioni di quantizzazione
    converter.optimizations = [tf.lite.Optimize.DEFAULT]
    # Imposta la funzione per il representative dataset
    converter.representative_dataset = representative_dataset_gen
    # Check sul fatto che il modello possa usare operazioni su interi a 8 bit
    converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
    # Converti il modello
    tflite_model = converter.convert()

    tflite_file_path = "yolov5s-int8.tflite"
    with open(tflite_file_path, "wb") as f:
        f.write(tflite_model)

    print(f"Modello TFLite INT8 creato: '{tflite_file_path}'")
    return tflite_file_path
```

Delegati - GPU

L'attivazione di questo GPUdelegate comporta un notevole aumento della velocità di inferenza e un miglioramento del throughput. Uso la funzionalità "options" dell'interprete.

```java
// Configura le opzioni dell'interprete
Interpreter.Options options = new Interpreter.Options();

// Aggiungi il delegato GPU
GpuDelegate gpuDelegate = new GpuDelegate();
options.addDelegate(gpuDelegate);

// Crea l'interprete TensorFlow Lite e assegna il delegato GPU
tflite = new Interpreter(loadModelFile(assetManager, modelPath), options);
```

Rilevazione di Anomalie - Addestramento

```python
# Utilizza solo le immagini "good" per addestramento
train_dir = 'C:/Users/andre/Downloads/mvtec_anomaly_detection/grid/train/good'
X_train_normal = prepare_data(train_dir)
```

```python
# Crea e compila il modello
autoencoder = create_autoencoder(input_shape)
autoencoder.compile(optimizer='adam', loss='mean_squared_error')
```

```python
# 3. Addestramento del Modello
print("Inizio addestramento del modello...")
history = autoencoder.fit(X_train_normal, X_train_normal,
                          epochs=50,
                          batch_size=32,
                          shuffle=True,
                          validation_split=0.1) # Usa il 10% dei dati per la validazione
print("Addestramento completato.")
```

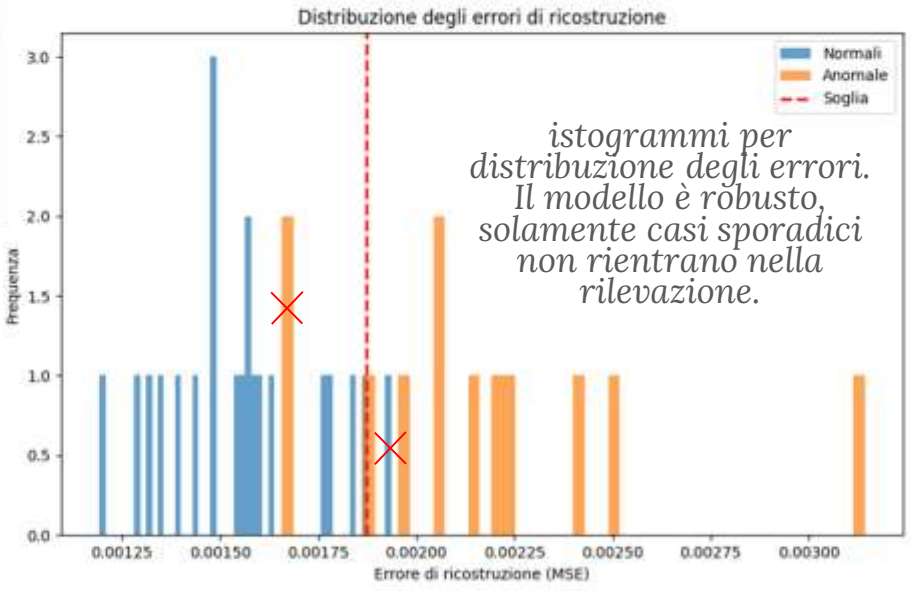
Esempio di addestramento con cento epoche

```
Epoch 85/100
15/15 ━━━━━━━━━━━━━━━━━━━━ 5s 351ms/step - loss: 0.0021 - val_loss: 0.0022
Epoch 86/100
15/15 ━━━━━━━━━━━━━━━━━━━━ 5s 343ms/step - loss: 0.0021 - val_loss: 0.0021
Epoch 87/100
15/15 ━━━━━━━━━━━━━━━━━━━━ 5s 351ms/step - loss: 0.0021 - val_loss: 0.0021
Epoch 88/100
15/15 ━━━━━━━━━━━━━━━━━━━━ 5s 344ms/step - loss: 0.0021 - val_loss: 0.0020
Epoch 89/100
15/15 ━━━━━━━━━━━━━━━━━━━━ 5s 346ms/step - loss: 0.0020 - val_loss: 0.0020
Epoch 90/100
15/15 ━━━━━━━━━━━━━━━━━━━━ 5s 345ms/step - loss: 0.0021 - val_loss: 0.0020
Epoch 91/100
15/15 ━━━━━━━━━━━━━━━━━━━━ 5s 346ms/step - loss: 0.0020 - val_loss: 0.0020
Epoch 92/100
15/15 ━━━━━━━━━━━━━━━━━━━━ 5s 347ms/step - loss: 0.0020 - val_loss: 0.0021
Epoch 93/100
15/15 ━━━━━━━━━━━━━━━━━━━━ 5s 349ms/step - loss: 0.0020 - val_loss: 0.0022
```

Rilevazione di Anomalie - Test

```python
# Carica i dati di test (immagini normali e anomale)
test_dir_normal = 'C:/Users/andre/Downloads/mvtec_anomaly_detection/grid/test/good'
test_dir_anomalies = 'C:/Users/andre/Downloads/mvtec_anomaly_detection/grid/test/broken'
X_test_normal = prepare_data(test_dir_normal)
X_test_anomalies = prepare_data(test_dir_anomalies)

# Previsione sulle immagini di test
reconstructed_normal = autoencoder.predict(X_test_normal)
reconstructed_anomalies = autoencoder.predict(X_test_anomalies)
```



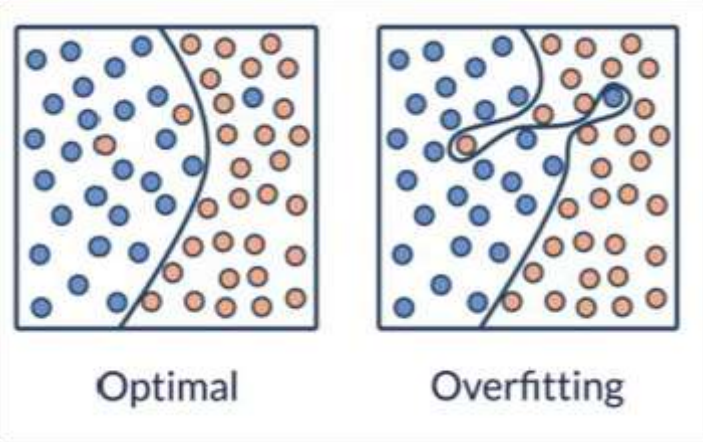
istogrammi per distribuzione degli errori. Il modello è robusto, solamente casi sporadici non rientrano nella rilevazione.

Sommario delle Soglie di Valutazione, nel complesso l'applicazione è robusta.

```
Errore MSE medio per immagini normali: 0.001559
Errore MSE medio per immagini anomale: 0.002162
Soglia di anomalia MSE (95° percentile): 0.001872
```

Rilevazione di Anomalie - Ottimizzazioni

Se il modello viene addestrato per troppe epoche, tende a memorizzare il set di dati di addestramento invece di imparare a generalizzare.



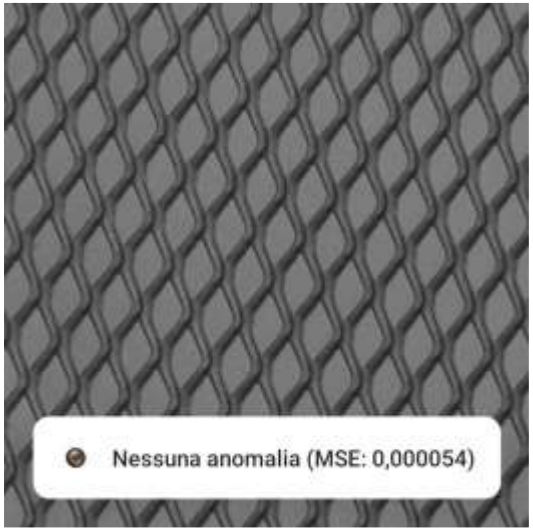
In generale, se la curva della perdita di validazione inizia a salire mentre la perdita di addestramento continua a scendere, il modello sta iniziando a sovra-adattarsi.

```python
# 3. Addestramento del Modello
print("Inizio addestramento del modello...")
history = autoencoder.fit(X_train_normal, X_train_normal,
                          epochs=50,
                          batch_size=32,
                          shuffle=True,
                          validation_split=0.1) # Usa il 10% dei dati per la validazione
print("Addestramento completato.")
```

Grazie a ciò il modello è robusto

person (
bicycle (0

Inferenza Modello
Scatta foto
Trova foto
Classificazione Immagini
Threshold: 0,55
Classifica
Salva immagine
Rilevamento di Anomalie
Soglia MSE: 0,001872
Detection anomalie


Nessuna anomalia (MSE: 0,000054)

Conclusioni e Sviluppi Futuri

- *In conclusione, la stesura dimostra che LiteRT ha potenziale enorme per innovazioni future in manutenzione predittiva, logistica e controllo qualità. Permette l'inferenza diretta su dispositivi edge con alta precisione, offrendo vantaggi come latenza ridotta, privacy e migliore efficienza.*
- *In futuro, questa tecnologia è destinata a trasformare i processi industriali, rendendoli più agili e sostenibili. L'evoluzione di runtime come LiteRT favorirà l'integrazione dell'AI, creando sistemi intelligenti più veloci, sicuri e robusti, anche in piattaforme complesse come Android.*

- *In linea con ciò, per quanto riguarda l'applicazione, una possibile direzione sul futuro è quella di raffinare ancora di più i dataset utilizzati e perfezionare gli addestramenti, consentendo al modello finale di essere preciso e specifico.*
- ***Visione Artificiale Specializzata:** è possibile specializzare il modello non solo sulle ottanta classi standard di Yolov5. L'allenamento verrà fatto su immagini specifiche a discrezione dell'utente. Un esempio è quello di integrare immagini di segnaletica e linee stradali, per gettare le basi di un'applicazione di guida autonoma su dispositivo periferico.*
- ***Rilevamento Anomalie con Heatmap Dinamiche:** è possibile inserire una heatmap di calore per il rilevamento di anomalie. Questa tecnica permetterà di calcolare dinamicamente i frame dell'immagine che si discostano maggiormente dal modello addestrato, offrendo una localizzazione visiva e precisa dell'anomalia.*